

```
# First cell - imports
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib_venn import venn2
import seaborn as sns
import numpy as np
```

Introduction

This study sought to elucidate the effect of transcription factor RUNX1 on chromatin organization and gene regulation in breast cancer cells. RUNX1 is known to be an oncogene and tumor suppressor however there is a gap in knowledge for how RUNX1 modulates gene regulation in a 3-dimensional setting, adding another level to its ability to regulate genetic expression. To answer this question they performed ChIP-Seq to map the binding of RUNX1, RNA-seq to observe changes in gene expression, and Hi-C to observe changes in the 3-dimensional organization of the genome. Further understanding of the impact of RUNX1 on chromatin organization on the gene expression of breast cancer cells will help to further research in the genomics and cancer fields to create therapies for patients.

Methods

Pipeline Overview

RUNX1 ChIP-seq data were processed using Nextflow v25.04.6. Singularity containers were used for reproducibility. The pipeline processed two biological replicates of RUNX1 ChIP-seq (IP_rep1 and IP_rep2) and corresponding input controls (INPUT_rep1 and INPUT_rep2).

Quality Control and Read Trimming

Raw sequencing reads were assessed for quality using FastQC v0.12.1. Adapter sequences and low-quality bases were trimmed using Trimmomatic v0.39 with default parameters. Quality control metrics across all samples were aggregated using MultiQC v1.15.

Read Alignment

Following quality trimming, reads were aligned to the human reference genome (GRCh38/hg38) using Bowtie2 v2.4.5 with default parameters. A Bowtie2 index was built from the GRCh38 primary assembly genome sequence prior to

alignment. SAMtools v1.21 was used to convert aligned reads to BAM format, sorted by genomic coordinate using SAMtools sort, and indexed using SAMtools index. Alignment statistics were generated using SAMtools flagstat.

Signal Track Generation and Visualization

Genome-wide coverage tracks were generated from sorted BAM files using deepTools bamCoverage v3.5.5, producing normalized bigWig files for visualization. Signal enrichment profiles were computed using deepTools computeMatrix and visualized using deepTools plotProfile. Sample correlation analysis was performed using deepTools multiBigwigSummary followed by deepTools plotCorrelation to assess replicate concordance.

Peak Calling

ChIP-seq peaks were called using HOMER v4.11 findPeaks. Tag directories were created from sorted BAM files using HOMER makeTagDirectory. Peak calling was performed independently for each biological replicate using the corresponding input control as background. HOMER findPeaks was run with the following parameters: FDR threshold = 0.001, fold enrichment over input greater than or equal to 4, Poisson p-value threshold = 1.00e-04, local fold enrichment greater than or equal to 4 over a 10 kb region, and local Poisson p-value threshold = 1.00e-04.

Identification of Reproducible Peaks

To identify high-confidence peaks present in both replicates, peak files were converted from HOMER format to BED format using HOMER pos2bed.pl. Reproducible peaks were identified using BEDTools intersect v2.31.1, requiring at least 1 bp overlap between replicates. Peaks overlapping ENCODE blacklist regions were removed using BEDTools subtract.

Peak Annotation

Reproducible peaks were annotated to genomic features using HOMER annotatePeaks.pl with the GENCODE v45 primary assembly annotation and the GRCh38 primary assembly genome sequence.

Motif Discovery

De novo motif discovery was performed on reproducible peaks using HOMER findMotifsGenome.pl to identify enriched transcription factor binding motifs

within RUNX1 binding sites.

Functional Enrichment Analysis

Unique gene names were extracted from the annotated peak file and used for functional enrichment analysis. Gene ontology and pathway enrichment were performed using Enrichr. Statistical significance was determined using Fisher's exact test with Benjamini-Hochberg FDR correction (adjusted p-value < 0.05).

Integration with RNA-seq Data

To assess the regulatory relationship between RUNX1 binding and gene expression, annotated ChIP-seq peaks were overlapped with lists of differentially expressed genes from RNA-seq analysis. Peaks were categorized by genomic location (within +/- 5 kb of TSS or within +/- 20 kb of gene bodies) and by gene expression status (up-regulated or down-regulated). The proportion of differentially expressed genes with RUNX1 binding at each genomic location was calculated and visualized.

Quality Control

Examining our MultiQC report we see that the general statistics section tells us a few key attributes of our data to keep in mind. First off, all of the reads have similar mapped amounts other than the control INPUT_rep2 with less than half of the reads of the other samples (10.2 million reads). It seems that something could have gone wrong in sample collection or when generating reads but since it is used for control/background it can be forgiven that it has less reads than the other samples. Another attribute we notice in the general statistics is that there are a high number of duplicates in the experimental samples IP_rep1 and IP_rep2 (89.1% and 74.3%). This is also something to keep an eye on but may be an artifact of working with ChIP-Seq data due to the fact that we are looking at specific regions that are biased to be repeated such as the transcription factor RUNX1

Next in our report we see the flagstat: read count section which tells us that what we discussed earlier, that there was a good amount of reads and alignment for each of the samples other than INPUT_rep2.

Trimmomatic shows us that we dropped a few million reads per sample, with more reads being dropped in the experimental group. This is normal and nothing to be concerned about.

Next we see the FastQC sequence counts report which shows us that there is very high duplication rates for our experimental group. This is something to keep an eye on and be concerned about but as we stated earlier, it may be due

to the fact that we are looking at ChIP-seq data of a specific transcription factor. We then see the mean quality scores graph which show us a slight dip in the front and back end of the reads, which is completely to be expected in a normal experiment. These reads all maintain a score above 30 and therefore pass. The per sequence quality score graph depicts the quality of reads to catch outlier groups, however the results are nominal. The next two graphs depict GC content and show that there is a slight difference between the experimental and control groups with the control group actually failing the check. However we might expect the GC and nucleotide content to differ between the groups as with the control group we would expect there to be a close to random distribution of nucleotides/GC content while with the experimental group we are looking at specifically where RUNX1 is binding so it will definitely bias the GC content. The per base N content shows little missing base calls.

Next in the report we see a sequence duplication levels report depicting high levels of duplication for IP_rep1 and IP_rep2. Again this can be explained as normal because the duplication arises from the fact that we are looking at RUNX1 binding sites which are bound to be more duplicated because they are a small subset of sequences. The opposite is true for the control group because they are meant to be random background. The same is shown in the overrepresented sequences sample summary, but this is expected. We additionally see a small number of over-represented sequences that are presumably from contamination during sample processing. Due to the small amount of contamination, we can safely ignore it for now.

Overall I can safely say that the experiment was of generally high-quality and is suitable for downstream analysis. This is because any of the fails we saw during the quality check can be attributed to the structure of the experiment with the other minor details such as low read counts in the control sample being negligible.

Spearman Correlation Distance between Correlation Coefficients:

I chose Spearman Correlation because ChIP-seq data is usually not normally distributed, Spearman can handle non-linear relationships caused by seq depth differences, and PCR amp bias. Spearman can also handle extreme values better from high read counts at strong binding sites. My correlation plot did not show as much of a difference between the experimental groups and control as the original paper however it still shows separation.

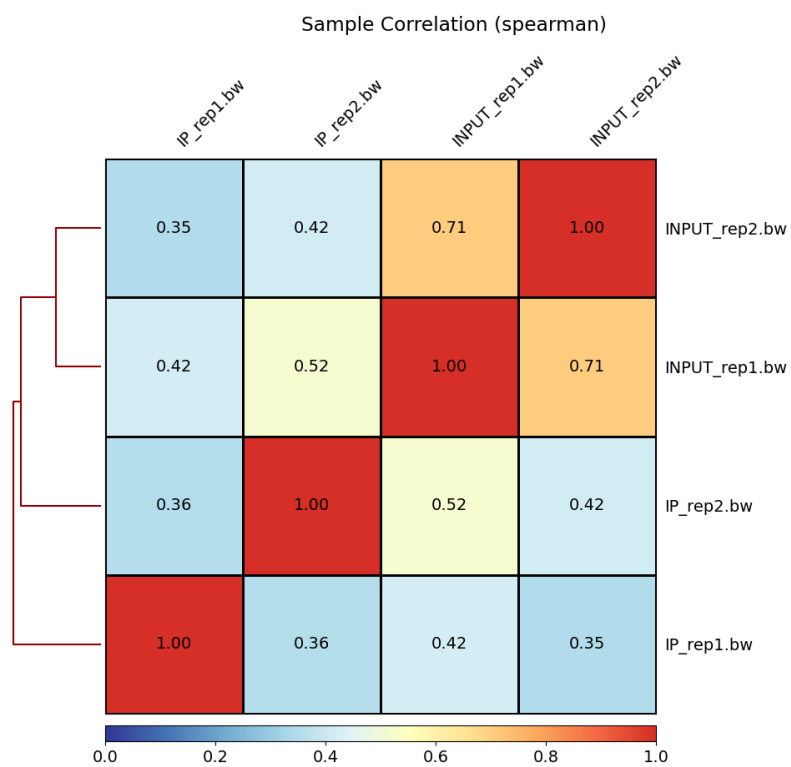


Figure 1: Gene Body Profile

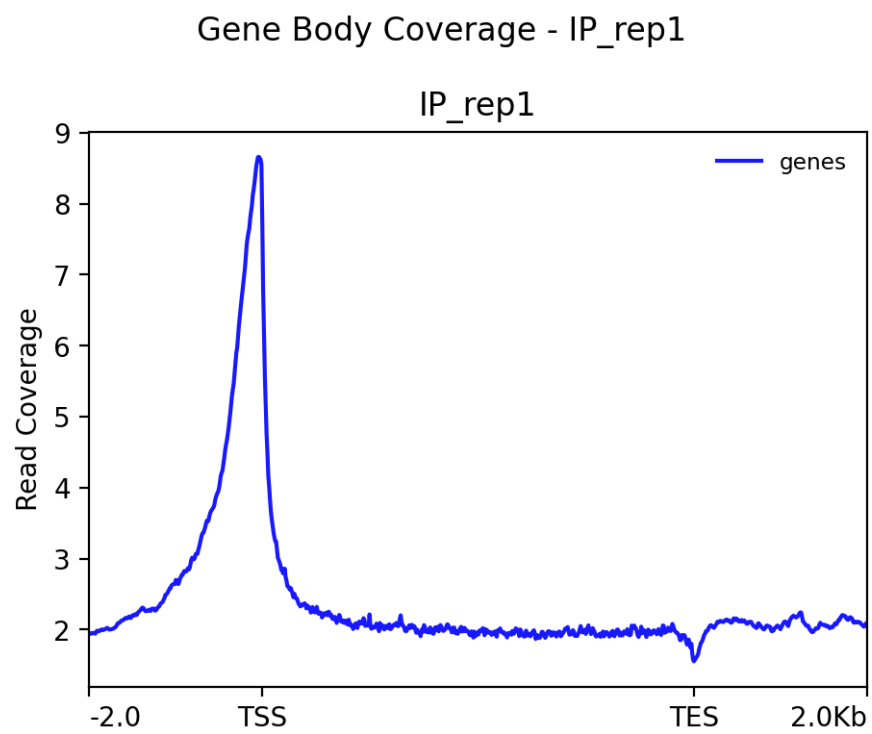


Figure 2: Gene Body Profile

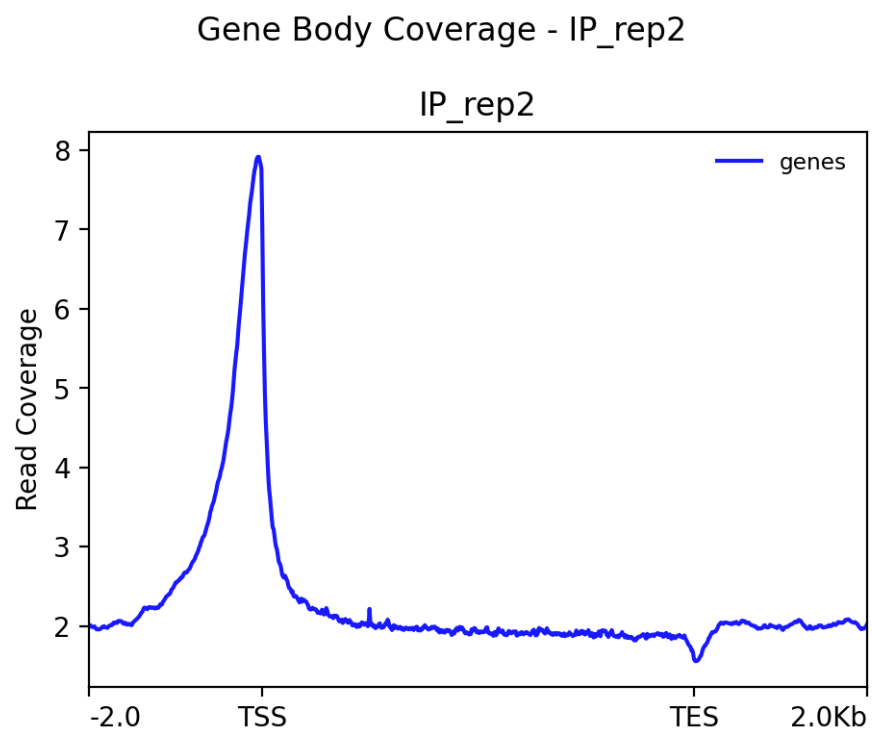


Figure 3: Gene Body Profile

GENE PROFILES IP_REP1/2:

BEDTOOLS INTERSECT REPRODUCIBLE PEAKS:

I chose a reciprocal overlap approach that looks for a minimum of 50% of overlap between features in file A and file B (-f 0.50) and making the 50% overlap reciprocal, going in both directions (-r). This approach filters out false positives and artifacts while the reciprocal nature prevents biases in peak differences between samples.

MOTIFS:

```
motifs = pd.read_csv('results/motif_enrichment/motif_output/knownResults.txt', sep='\t')
motifs.head(10)
```

Motif Name

Consensus

P-value

Log P-value

q-value (Benjamini)

of Target Sequences with Motif(of 5027)

% of Target Sequences with Motif

of Background Sequences with Motif(of 41145)

% of Background Sequences with Motif

0

RUNX(Runt)/HPC7-Runx1-ChIP-Seq(GSE22178)/Homer

SAAACCACAG

0.000000e+00

-1494.0

0.0

1465.0

29.14%

2135.4

5.19%

1

RUNX1(Runt)/Jurkat-RUNX1-ChIP-Seq(GSE29180)/Homer

AAACCACARM

0.000000e+00

-1142.0

0.0

1511.0

30.06%

3027.9

7.36%

2

RUNX2(Runt)/PCa-RUNX2-ChIP-Seq(GSE33889)/Homer

NWAACCACADNN

1.000000e-308

-709.7

0.0

1098.0

21.84%

2432.1

5.91%

3

RUNX-AML(Runt)/CD4+-PolII-ChIP-Seq(Barski_et_a...

GCTGTGGTTW

1.000000e-278

-640.6

0.0

1027.0

20.43%

2329.5

5.67%

4

Fra2(bZIP)/Striatum-Fra2-ChIP-Seq(GSE43429)/Homer

GGATGACTCATC

1.000000e-149

-345.3

0.0

504.0

10.03%

1015.9

2.47%

5

Fra1(bZIP)/BT549-Fra1-ChIP-Seq(GSE46166)/Homer

NNATGASTCATH

1.000000e-149

-345.0

0.0

523.0

10.40%

1090.9

2.65%

6

Fosl2(bZIP)/3T3L1-Fosl2-ChIP-Seq(GSE56872)/Homer

NATGASTCABNN

1.000000e-149

-344.0

0.0

428.0

8.51%

738.6

1.80%

7

Fos(bZIP)/TSC-Fos-ChIP-Seq(GSE110950)/Homer

NDATGASTCAYN

1.000000e-144
 -333.3
 0.0
 534.0
 10.62%
 1167.0
 2.84%
 8
 GRHL2(CP2)/HBE-GRHL2-ChIP-Seq(GSE46194)/Homer
 AAACYKGTTWDACMRGTTTB
 1.000000e-144
 -333.3
 0.0
 461.0
 9.17%
 882.6
 2.15%
 9
 JunB(bZIP)/DendriticCells-Junb-ChIP-Seq(GSE360...
 RATGASTCAT
 1.000000e-140
 -324.3
 0.0
 519.0
 10.32%
 1132.4
 2.75%

The first 4 results are all RUNX1 family which makes sense as this experiment is of a RUNX1 ChIP-seq. The enrichment of the Fos family suggests RUNX1 interacts with these AP-1 transcription factors which are involved in proliferation, differentiation, and stress response in hematopoietic cells. The further enrichment of JunB, another AP-1 transcription factor, point to RUNX1

working with these other transcription factors to regulate gene expression in differentiation and development of progenitor cells. These findings are very useful mechanistically speaking as well as therapeutically.

Genomic Tracks: MALAT1 and NEAT1

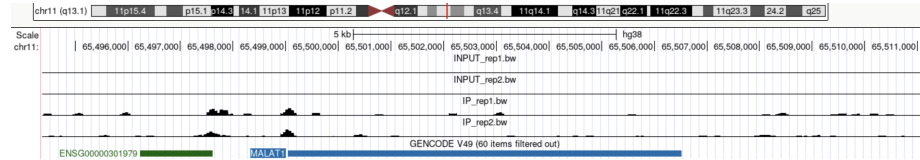


Figure 4: Screenshot 2025-11-26 at 2.10.33 AM.png

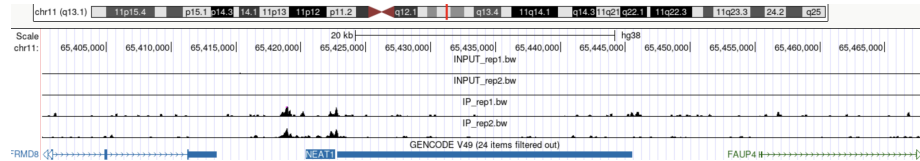


Figure 5: Screenshot 2025-11-26 at 2.17.29 AM.png

From my peaks I observe, when comparing to the INPUT control replicates, significant peaks at the promoter region for both the MALAT1 and NEAT1 genes. The peaks are not identical to the paper but replicate the same pattern. Differences observed could be entirely due to scaling of the graphs or from a difference in bin sizes when generating the bigWig and normalization.

RNAseq RUNX1

```
rnaseq = pd.read_csv('results/GSE75070_MCF7_shRUNX1_shNS_RNAseq_log2_foldchange.txt', sep=
```

Upregulated Genes:

```
rnaseq.query('padj < .01 & log2FoldChange > 1')
```

genename

transcript

log2FoldChange

padj

48

SGOL1
 NM_001012409,NM_001012410,NM_001012411,NM_0010...
 1.865109
 7.048718e-16
 50
 MYCN
 NM_001293228,NM_001293231,NM_001293233
 1.961676
 9.265427e-05
 66
 INCENP
 NM_001040694,NM_020238
 1.635607
 3.296986e-14
 80
 VCAN
 NM_001126336,NM_001164097,NM_001164098,NM_004385
 2.010163
 1.561000e-06
 86
 TPI1
 NM_001159287,NM_001258026
 1.052948
 9.799512e-11
 ...
 ...
 ...
 ...
 ...
 15256
 PCDHAC2

NM_018899,NM_031883
 1.590426
 1.305452e-07
 15274
 TCF19
 NM_001077511,NM_007109
 1.684759
 1.205806e-18
 15282
 NEK2
 NM_001204182,NM_001204183,NM_002497
 1.349700
 1.898691e-05
 15335
 SMAD9
 NM_001127217,NM_005905
 1.333194
 8.277110e-03
 15374
 MAP1B
 NM_005909
 1.616677
 3.342320e-09
 687 rows $\times$ 4 columns

Downregulated Genes:

```

rnaseq.query('padj < .01 & log2FoldChange < -1')
genename
transcript
log2FoldChange

```

padj
 39
 CYSRT1
 NM_199001
 -1.179227
 6.910866e-04
 139
 AGAP9
 NM_001190810,NM_001190810_chr10:47191843_47213626
 -1.179856
 2.622705e-04
 146
 TM2D1
 NM_032027
 -1.120710
 2.415048e-07
 162
 SEMA4C
 NM_017789
 -1.480403
 4.369624e-13
 206
 CCDC78
 NM_001031737
 -1.080449
 1.883375e-03
 ...
 ...
 ...
 ...
 ...

```

15330
UCKL1-AS1
NR_027287
-1.222779
2.415494e-04
15343
SCARNA12
NR_003010
-1.013360
2.225647e-08
15367
TM4SF1
NM_014220
-2.659629
1.184314e-13
15381
PAN2
NM_001127460,NM_001166279,NM_014871
-1.394833
1.368096e-28
15388
NYAP2
NM_020864
-1.432949
4.774104e-03
466 rows × 4 columns
DE_Genes = rnaseq.query('padj < .01 & abs(log2FoldChange) > 1')['genename'].tolist()
POS_Genes = rnaseq.query('padj < .01 & log2FoldChange > 1')['genename'].tolist()
NEG_Genes = rnaseq.query('padj < .01 & log2FoldChange < -1')['genename'].tolist()

```


Annotated Peaks

```
ap = pd.read_csv('results/peak_annotation/annotated_peaks.txt', sep='\t')
```

```
ap.head(10)
```

```
PeakID (cmd=annotatePeaks.pl intersect__peaks__filtered.bed GRCh38.primary__assembly.genome.fa  
-gtf gencode.v45.primary__assembly.annotation.gtf -annStats annota-  
tion_stats.txt)
```

Chr

Start

End

Strand

Peak Score

Focus Ratio/Region Size

Annotation

Detailed Annotation

Distance to TSS

Nearest PromoterID

Entrez ID

Nearest Unigene

Nearest Refseq

Nearest Ensembl

Gene Name

Gene Alias

Gene Description

Gene Type

0

chr20-1

chr20

47319843

47320056

+

763.5
 NaN
 promoter-TSS (ENST00000468376.2)
 protein_coding-promoter-TSS (ENST00000468376.2)
 -888
 ENST00000468376.2
 ENSG00000101040
 ENST00000360911
 NaN
 NaN
 ZMYND8
 NaN
 NaN
 protein_coding
 1
 chr20-2
 chr20
 57264338
 57264529
 +
 565.3
 NaN
 intron (ENST00000450594.6, intron 1 of 5)
 protein_coding-intron (ENST00000395863.8, intron 1 of 5)
 1345
 ENST00000433911.1
 ENSG00000101144
 ENST00000395863
 NaN
 NaN
 BMP7

NaN
NaN
protein_coding
2
chr17-4
chr17
61568691
61568928
+
511.4
NaN
Intergenic
Intergenic
22410
ENST00000521764.3
ENSG00000253506
ENST00000521764
NaN
NaN
NACA2
NaN
NaN
protein_coding
3
chr16-1
chr16
11915972
11916193
+
506.3
NaN

promoter-TSS (ENST00000420576.6)
 protein_coding-promoter-TSS (ENST00000420576.6)
 0
 ENST00000420576.6
 ENSG00000103342
 ENST00000439887
 NaN
 NaN
 GSPT1
 NaN
 NaN
 protein_coding
 4
 chr17-3
 chr17
 61231088
 61231269
 +
 500.0
 NaN
 intron (ENST00000587294.5, intron 1 of 4)
 protein_coding-intron (ENST00000390652.9, intr...
 -14038
 ENST00000592702.5
 ENSG00000141376
 ENST00000587294
 NaN
 NaN
 BCAS3
 NaN
 NaN

protein_coding
 5
 chr1-2
 chr1
 205211626
 205211835
 +
 485.9
 NaN
 promoter-TSS (ENST00000367161.7)
 protein_coding-promoter-TSS (ENST00000367161.7)
 -28
 ENST00000367162.8
 ENSG00000133059
 ENST00000367162
 NaN
 NaN
 DSTYK
 NaN
 NaN
 protein_coding
 6
 chr15-1
 chr15
 50355255
 50355432
 +
 476.6
 NaN
 promoter-TSS (ENST00000220429.12)
 protein_coding-promoter-TSS (ENST00000220429.12)

15
 ENST00000560359.1
 ENSG00000244879
 ENST00000561289
 NaN
 NaN
 GABPB1-AS1
 NaN
 NaN
 lncRNA
 7
 chr5-1
 chr5
 178078279
 178078500
 +
 422.6
 NaN
 intron (ENST00000651413.2, intron 14 of 14)
 lncRNA-intron (ENST00000651413.2, intron 14 of. . .
 13028
 ENST00000651483.1
 ENSG00000204677
 ENST00000652167
 NaN
 NaN
 FAM153CP
 NaN
 NaN
 lncRNA
 8

chr2-1
 chr2
 11498623
 11498856
 +
 412.8
 NaN
 intron (ENST00000628795.2, intron 1 of 2)
 protein_coding-intron (ENST00000628795.2, intr...
 16398
 ENST00000628795.2
 ENSG00000196208
 ENST00000389825
 NaN
 NaN
 GREB1
 NaN
 NaN
 protein_coding
 9
 chr17-2
 chr17
 59964527
 59964756
 +
 398.8
 NaN
 promoter-TSS (ENST00000305783.13)
 protein_coding-promoter-TSS (ENST00000305783.13)
 71
 ENST00000586083.5

ENSG00000189050

ENST00000466544

NaN

NaN

RNFT1

NaN

NaN

protein_coding

Annotated Peaks with Differentially Expressed Genes

```
DE_Peaks = ap.query('Gene Name' in @DE_Genes')
POS_Peaks = ap.query('Gene Name' in @POS_Genes')
NEG_Peaks = ap.query('Gene Name' in @NEG_Genes')

up_5kb = POS_Peaks[(POS_Peaks['Distance to TSS'] < 5000) & (POS_Peaks['Distance to TSS'] > -5000)]
down_5kb = NEG_Peaks[(NEG_Peaks['Distance to TSS'] < 5000) & (NEG_Peaks['Distance to TSS'] > 5000)]

up_20kb = POS_Peaks[(POS_Peaks['Distance to TSS'] < 20000) & (POS_Peaks['Distance to TSS'] > -20000)]
down_20kb = NEG_Peaks[(NEG_Peaks['Distance to TSS'] < 20000) & (NEG_Peaks['Distance to TSS'] > 20000)]

data = {
    'Upregulated': [up_5kb, up_20kb],
    'Downregulated': [down_5kb, down_20kb]
}

# Define the row labels
index = ['±5kb', '±20kb']

# Create DataFrame
table = pd.DataFrame(data, index=index)

print(table)
```

	Upregulated	Downregulated
±5kb	73	58
±20kb	97	78

Number of Peaks After Filter

```
%%bash
wc -l results/homer/peaks/IP_rep1_peaks.txt
```



```

wc -l results/homer/peaks/IP_rep2_peaks.txt
wc -l results/reproducible_peaks/reproducible_peaks.bed
wc -l results/filtered_peaks/intersect_peaks_filtered.bed

89400 results/homer/peaks/IP_rep1_peaks.txt 22937 results/homer/peaks/IP_rep2_peaks.txt
5050 results/reproducible_peaks/reproducible_peaks.bed 5027 results/filtered_peaks/intersect_peaks_filtered.b

```

Annotation

```

DE_Promoters = DE_Peaks[DE_Peaks['Annotation'].str.contains('promoter-TSS')]['Gene Name'].to

with open('results/DE_Promoters.txt', 'w') as w:
    w.write('\n'.join(DE_Promoters))

print(DE_Promoters)

['PKM', 'ABHD17C', 'COMMD2', 'THEM4', 'TBL1X', 'ATAD2', 'HELLS',
'FMN1', 'MCM7', 'ARID1A', 'MALAT1', 'CS', 'RAD52', 'USPL1', 'ASNS',
'TFF1', 'ALDOA', 'STAG3L4', 'CAPN8', 'SYBU', 'ZNF224', 'PAN2', 'RFC3',
'CALM3', 'DHTKD1', 'ARID1A', 'FRA10AC1', 'CNST', 'GOLGA8B',
'LGALS1', 'MALAT1', 'CHAC1', 'WIPF1', 'SCD', 'PRSS16', 'MCM7',
'RACGAP1', 'GINS2', 'BUB1', 'KRT7', 'FAH', 'ATXN2L', 'PLK1', 'CKAP2L',
'KIFC1', 'GPHN', 'PGD', 'CENPU', 'PNN', 'FDPS', 'GDF15', 'HSPB8',
'ISOC1', 'PLCH1', 'HSPB8', 'PTTG1', 'PRDM6', 'PLEKHG5', 'PCBP2',
'PCK2', 'C1orf53', 'PRPF3', 'ATIC', 'CCNA2', 'GPNMB', 'TOB2', 'IQSEC1',
'INPP5D', 'PLA2G6', 'POC1A', 'NEIL3', 'KSR2', 'TM2D1', 'FAM151B',
'MPND', 'TACC1', 'ANP32E', 'RFC4', 'INCENP', 'PKMYT1', 'TMPRSS2',
'HEMK1', 'C6orf141', 'SYNM', 'TVP23B', 'GAPDH', 'C1GALT1C1', 'CTNNA3',
'ZNF75D', 'C5orf34', 'KLHL17', 'TUBB2A', 'EPHA4', 'APOE']

fp = ap[ap['Gene Name'].isin(DE_Promoters)]

total_up_genes = len(POS_Genes) # Total up-regulated genes
total_down_genes = len(NEG_Genes) # Total down-regulated genes

# For +/- 5kb of TSS
up_5kb_bound = POS_Peaks[(POS_Peaks['Distance to TSS'] < 5000) &
                          (POS_Peaks['Distance to TSS'] > -5000)]['Gene Name'].nunique()
down_5kb_bound = NEG_Peaks[(NEG_Peaks['Distance to TSS'] < 5000) &
                            (NEG_Peaks['Distance to TSS'] > -5000)]['Gene Name'].nunique()

# For "whole gene" - peaks that are NOT intergenic
up_gene_bound = POS_Peaks[
    ~POS_Peaks['Annotation'].str.contains('Intergenic', case=False, na=False)
]['Gene Name'].nunique()

```

```

down_gene_bound = NEG_Peaks[
    ~NEG_Peaks['Annotation'].str.contains('Intergenic', case=False, na=False)
]['Gene Name'].nunique()

# Calculate not bound using the ORIGINAL gene list totals
up_5kb_notbound = total_up_genes - up_5kb_bound
down_5kb_notbound = total_down_genes - down_5kb_bound
up_gene_notbound = total_up_genes - up_gene_bound
down_gene_notbound = total_down_genes - down_gene_bound

# Prepare data for plotting
labels = ['Up-regulated', 'Down-regulated', 'Up-regulated', 'Down-regulated']
sublabels = ['+/- 5kb of TSS', '+/- 5kb of TSS', '+/- 20kb of whole gene', '+/- 20kb of whole gene']

runx2_bound = [up_5kb_bound, down_5kb_bound, up_gene_bound, down_gene_bound]
not_bound = [up_5kb_notbound, down_5kb_notbound, up_gene_notbound, down_gene_notbound]
totals = [total_up_genes, total_down_genes, total_up_genes, total_down_genes]

# Calculate percentages
not_bound_pct = [nb/tot * 100 for nb, tot in zip(not_bound, totals)]
runx2_bound_pct = [rb/tot * 100 for rb, tot in zip(runx2_bound, totals)]

# Create the plot
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(len(labels))
width = 0.6

fig, ax = plt.subplots(figsize=(10, 6))

p1 = ax.bar(x, not_bound_pct, width, color='gray', label='Not bound')
p2 = ax.bar(x, runx2_bound_pct, width, bottom=not_bound_pct, color='#FF6B35', label='RUNX1 bound')

# Add count labels
for i, (nb, rb) in enumerate(zip(not_bound, runx2_bound)):
    ax.text(i, not_bound_pct[i]/2, str(nb), ha='center', va='center',
            fontsize=11, fontweight='bold', color='white')
    ax.text(i, not_bound_pct[i] + runx2_bound_pct[i]/2, str(rb), ha='center', va='center',
            fontsize=11, fontweight='bold', color='white')

ax.set_ylabel('Percentage of genes', fontsize=12)
ax.set_ylim(0, 105)
ax.set_yticks(range(0, 101, 10))
ax.set_yticklabels([f'{i}%' for i in range(0, 101, 10)])
ax.set_xticks(x)

```

```

x_labels = [f'{label}\n{sublabel}' for label, sublabel in zip(labels, sublabels)]
ax.set_xticklabels(x_labels, rotation=0, ha='center')

ax.legend(loc='upper right')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

```

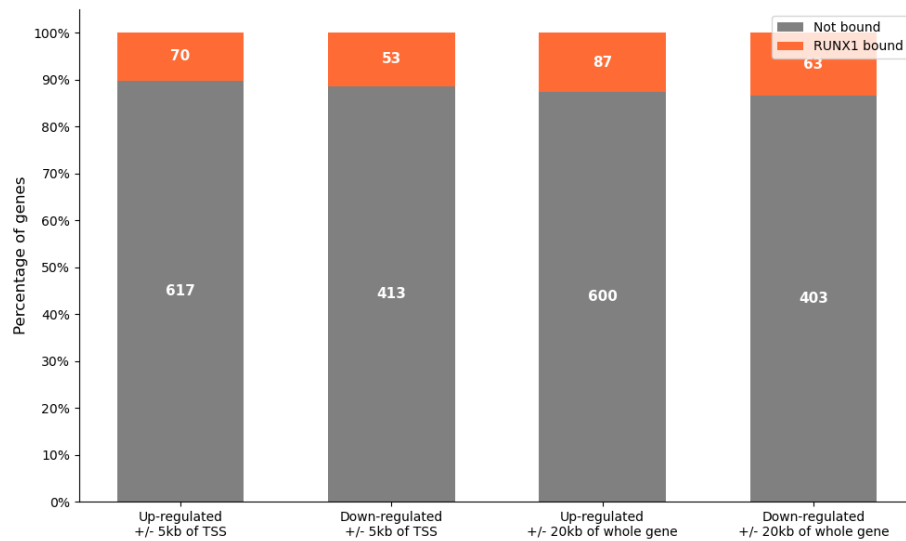


Figure 6: png

There are some differences between my results and that of the paper. Most notable differences are found in the +/- 20kb columns with the paper showing a much larger overlap of RUNX1 bound genes than I did. The total amount of genes for each column is consistent with the paper and the amount of overlap for the two first columns is comparable, however the paper depicts a much larger overlap in the amount of RUNX1 bound genes being up/downregulated in the 20kb category while my data only depicts a negligible increase in overlap. These differences can be explained by how the paper defines +/- 20kb of whole gene meaning if they took in a larger area around the gene they could pick up more overlap. Another way they can be explained is that the gene annotation for these peaks was done using the hg19 while we used hg38. Using a different gene annotation definitely had significant impact on the gene overlaps. Combining the analysis of both the 5kb and 20kb regions allows us to look at direct binding (5kb) and indirect effectors of the RUNX1 transcription factor. This would provide us with valuable information on off-target effects and clues into the

mechanism for how RUNX1 functions. It would also allow us to differentiate between direct effects of binding by RUNX1 and that of more distant bindings.

Sup Fig S2A Table

```
data = { #from multigc_report
  'Sample Name': ['RUNX1 ChIP 1', 'RUNX1 ChIP 2', 'INPUT 1', 'INPUT 2'],
  'Biological Replicate': [1, 2, 1, 2],
  'Raw Reads': [29700000, 30000000, 30100000, 10900000],
  'Mapped Reads': [27800000, 28200000, 28600000, 10100000]
}

df = pd.DataFrame(data)

# Create figure
fig, ax = plt.subplots(figsize=(12, 3))
ax.axis('tight')
ax.axis('off')

# Format the display data with commas
display_data = df.copy()
display_data['Raw Reads'] = display_data['Raw Reads'].apply(lambda x: f"{x:,}")
display_data['Mapped Reads'] = display_data['Mapped Reads'].apply(lambda x: f"{x:,}")

# Create table
table = ax.table(cellText=display_data.values,
                 colLabels=display_data.columns,
                 cellLoc='center',
                 loc='center',
                 bbox=[0, 0, 1, 1])

# Style the table
table.auto_set_font_size(False)
table.set_fontsize(12)
table.scale(1, 2.5)

# Style header row
for i in range(len(display_data.columns)):
    cell = table[(0, i)]
    cell.set_facecolor('#FFFFFF')
    cell.set_text_props(weight='bold', fontsize=13)
    cell.set_edgecolor('black')
    cell.set_linewidth(2)
```

```

# Style data cells
for i in range(1, len(display_data) + 1):
    for j in range(len(display_data.columns)):
        cell = table[(i, j)]
        cell.set_edgecolor('black')
        cell.set_linewidth(1.5)
        cell.set_facecolor('#FFFFFF')

# Add thicker borders for outer edges
for key, cell in table.get_celld().items():
    cell.set_linewidth(1.5)

plt.show()

```

Sample Name	Biological Replicate	Raw Reads	Mapped Reads
RUNX1 ChIP 1	1	29,700,000	27,800,000
RUNX1 ChIP 2	2	30,000,000	28,200,000
INPUT 1	1	30,100,000	28,600,000
INPUT 2	2	10,900,000	10,100,000

Figure 7: png

There are differences between the number of mapped and raw reads between my created table and the table found in the paper. The number of raw reads between the two figures is quite similar other than mine are rounded. The number of mapped reads are obviously different with my data showing millions of reads more mapped per sample. The rounding is just due to me pulling this data from the multiqc report. The difference in mapped reads between the samples can easily be explained due to my data being grabbed before the filtering step which would remove duplicates and low quality reads while their table was likely created with filtered data.

Sup Fig S2C Venn Diagram

```

# from peak_stats.txt of reproducible peaks
total_rep1 = 22897
total_rep2 = 89360
shared = 5050

# Calculate unique to each replicate

```

```

unique_rep1 = total_rep1 - shared # 17847
unique_rep2 = total_rep2 - shared # 84310

print(f"Unique to Rep1: {unique_rep1}")
print(f"Unique to Rep2: {unique_rep2}")
print(f"Shared: {shared}")

# Create figure
fig, ax = plt.subplots(figsize=(10, 8))

# Create Venn diagram
v = venn2(subsets=(unique_rep1, unique_rep2, shared),
          set_labels=('Rep1', 'Rep2'),
          ax=ax)

# Customize colors to match the paper
v.get_patch_by_id('10').set_color('#FF6B6B') # Rep1 only - red/coral
v.get_patch_by_id('01').set_color('#A8B5FF') # Rep2 only - blue
v.get_patch_by_id('11').set_color('#C77DFF') # Overlap - purple

# Adjust transparency
v.get_patch_by_id('10').set_alpha(0.7)
v.get_patch_by_id('01').set_alpha(0.6)
v.get_patch_by_id('11').set_alpha(0.7)

# Customize text - labels
for text in v.set_labels:
    text.set_fontsize(28)
    text.set_fontweight('bold')

# Customize text - numbers
for text in v.subset_labels:
    text.set_fontsize(22)
    text.set_fontweight('bold')

# Remove axis
ax.axis('off')

plt.show()

```

Unique to Rep1: 17847 Unique to Rep2: 84310 Shared: 5050

There are significant differences between my called peaks and that of the paper. For instance I have 17,847 and 84,310 called peaks for Rep1 and 2 respectively. Meanwhile the paper has only a fraction of this amount with 3,983 and 10,465 peaks for Rep1 and 2 respectively. The shared peaks is more similar with the original paper having 3,466 peaks to my 5,050. The discrepancies between my

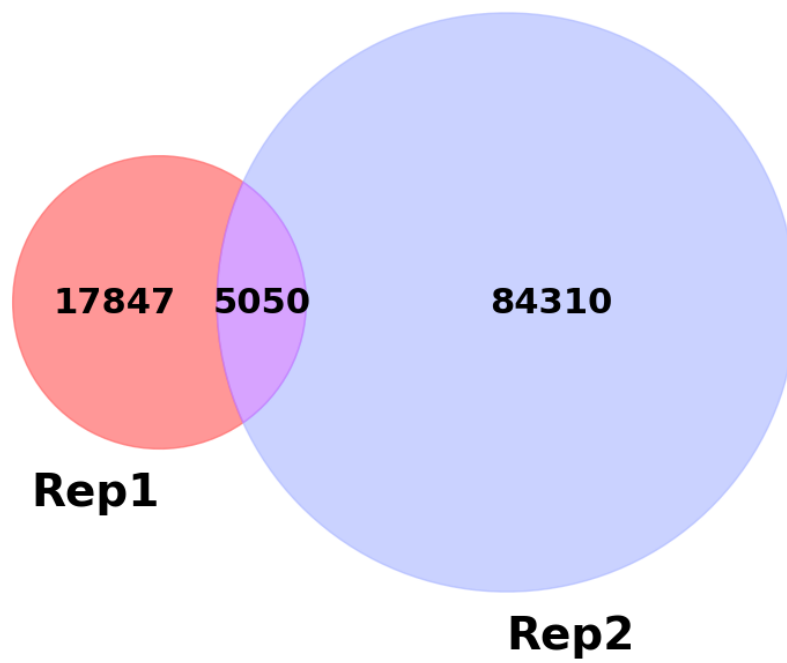


Figure 8: png

called peaks and that of the original paper likely has to do with the differences in how the data was filtered, as in the paper had more aggressive filtering when compared to my approach. Another difference could stem from how the paper called peaks. I called peaks with a regressive 50% overlap but the paper may have been more stringent.

Analyze Annotated Peaks

```
# Load annotated peaks file
peaks = pd.read_csv('results/peak_annotation/annotated_peaks.txt', sep='\t')

print(f"Total peaks: {len(peaks)}")

# Extract unique gene names
genes = peaks['Gene Name'].dropna().unique().tolist()

print(f"Total unique genes with RUNX1 binding: {len(genes)}")
print(f"First 10 genes: {genes[:10]}")

# Save gene list
with open('RUNX1_target_genes.txt', 'w') as f:
    f.write('\n'.join(genes))

Total peaks: 5027 Total unique genes with RUNX1 binding: 4368 First 10 genes:
['ZMYND8', 'BMP7', 'NACA2', 'GSPT1', 'BCAS3', 'DSTYK', 'GABPB1-AS1',
'FAM153CP', 'GREB1', 'RNFT1']

# Filter for promoter peaks
promoter_peaks = peaks[peaks['Annotation'].str.contains('promoter', case=False, na=False)]

# Extract unique promoter-bound genes
promoter_genes = promoter_peaks['Gene Name'].dropna().unique().tolist()

print(f"Total promoter-bound genes: {len(promoter_genes)}")
print(f"First 10 genes: {promoter_genes[:10]}")

# Save to text file
with open('RUNX1_promoter_bound_genes.txt', 'w') as f:
    f.write('\n'.join(promoter_genes))

Total promoter-bound genes: 2082 First 10 genes: ['ZMYND8', 'GSPT1',
'DSTYK', 'GABPB1-AS1', 'RNFT1', 'ENSG00000290230', 'AP4B1',
'LINC02875', 'PRKCI', 'NCOA3']
```

From the annotated peaks data I filtered for promoter-bound genes. These genes

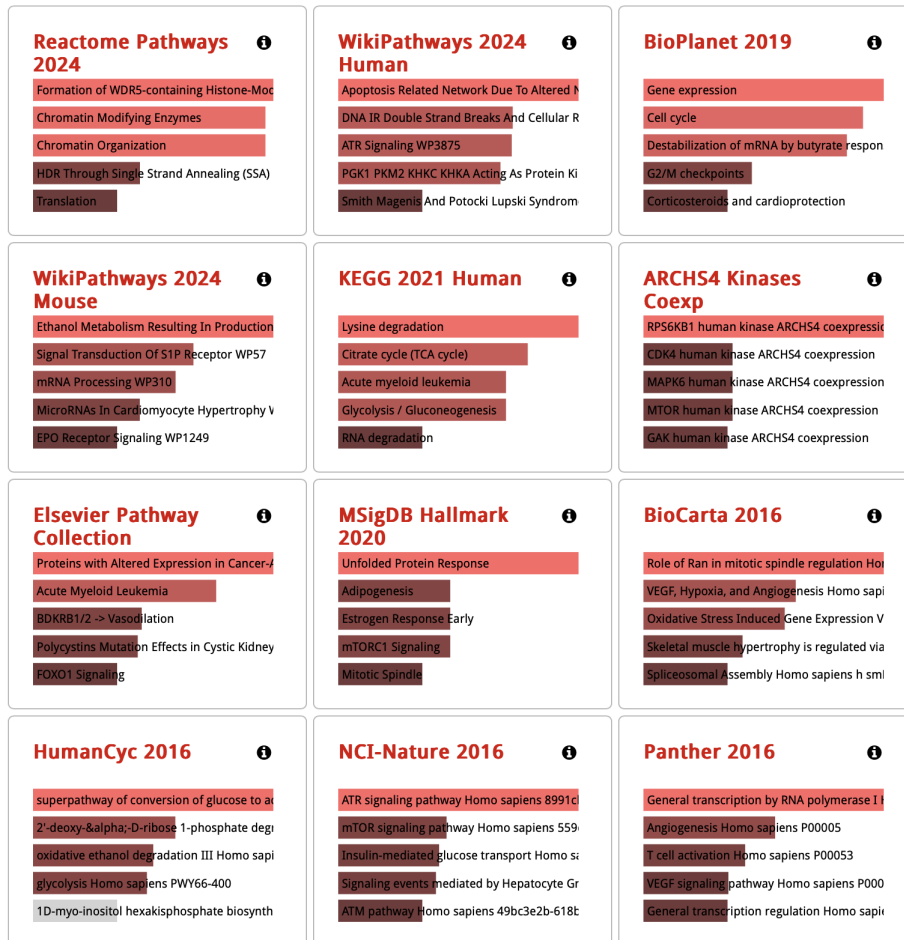


Figure 9: image.png

would show us direct transcriptional targets of RUNX1, removing any indirect regulatory relationships, and allowing us to focus on a few key pathways. We see in the reactome pathways 2024, a strong enrichment for histone modifications as well in chromatin modifying enzymes, and chromatin organization. This is notable because RUNX1 is a master transcription factor that utilizes chromatin remodeling complexes to regulate gene expression during cell development. In BioPlanet 2019 we see pathways for cell cycle and G2/M checkpoints which shows its direct role in cell cycle progression and its involvement in breast cancer development as an oncogene/tumor suppressor. This is further supported by Acute myeloid leukemia as seen in the KEGG 2021 Human pathway list. In this pathway we also see RUNX1's involvement in metabolic pathways; glycolysis, citrate cycle, and lysine degradation. Its involvement in coordinating cellular stress response (mTOR signaling) further elucidates its far reaching impact on gene expression and its utility in cancer treatment and research.